

ZERO-TO-JOB-READY DATA ANALYST 2026

M07 - PYTHON FOR DATA ANALYSTS

Jupyter -> pandas -> cleaning -> grouping -> merge -> reshape -> charts -> EDA

FRAME -> LOAD -> AUDIT -> PREPARE -> MERGE -> ANALYZE -> VALIDATE -> RERUN

Clear, reproducible analyst code beats clever code.

This book is your teacher

- Assume Python/Jupyter is new. The notebook/kernel/DataFrame mental model comes before code-heavy practice.
- Type the important analyst code yourself. Copying the whole teacher example creates familiarity, not independent skill.
- Every cleaning/merge/group operation keeps a validation path: before/after rows, key checks, unmatched counts or reconciled totals.
- Charts are built from validated summaries and saved as separate figures. EDA separates data-quality observations from business findings.
- Protected review opens only after a genuine attempt. Final delivery must survive Restart Kernel -> Run All with relative/reproducible paths.

Normal Python loop

1. Read question + mental model
2. Learn new Python/pandas words
3. Watch exact visual if useful
4. Follow teacher code once
5. Do guided notebook cell
6. Close support + changed cell
7. Validate counts/keys/totals
8. Explain + save + rerun

Contents / Index

PY0 Python, Jupyter and a reproducible analyst notebook	6
PY1 Practical Python basics: values, conditions, loops and functions	9
PY2 pandas: load, inspect, select and filter	12
PY3 Data quality and cleaning in pandas	15
PY4 GroupBy, aggregation and result grain	18
PY5 Merge, concat and validation	21
PY6 Reshape and time analysis	24
PY7 Visualization with pandas and Matplotlib	27
PY8 Exploratory data analysis (EDA)	30
PY9 Notebook quality, outputs and reproducibility	33
PY10 Integrated analyst workflow in Python	36
M-PY Integrated analyst workflow project	39
End-of-module mastery checklist	43
Revision / Python quick reference	44

Module prerequisite and tool map

0. Prerequisite

M00-M06 taught analyst reasoning, grain, cleaning, key/cardinality checks, grouped summaries and evidence. Python syntax is not assumed. The purpose here is to express those habits reproducibly in Python/pandas.

Tools / files

- Python 3 + Jupyter Notebook/JupyterLab; pandas and Matplotlib.
- C1_M07_PYTHON_FOR_ANALYSTS_CANONICAL_PRACTICE_NOTEBOOK_RC1.ipynb - 50 cells with exact PY0-PY10 mapping.
- Practice_Data/ - sales, products, customers, returns and region targets.
- C1_M07_PYTHON_FOR_ANALYSTS_PROTECTED_PRACTICE_REVIEW_RC1.pdf - post-attempt only.
- Gate_Data/ + Gate A/B/C notebooks + separate protected Gate reviews.
- outputs/ is created by the notebook; use relative paths, not machine-specific absolute paths.

Important route defect removed

The inherited notebook shifted PY3-PY9 practice labels/tasks away from the canonical lessons.

The RC1 notebook maps each PY0-PY10 lesson directly to its own guided + changed cell.

No old shifted heading is treated as canonical.

Exact practice, project and Gate route

- PY0 environment/reproducibility -> PY1 basics -> PY2 load/inspect/filter -> PY3 cleaning -> PY4 GroupBy -> PY5 merge -> PY6 reshape/time -> PY7 Matplotlib -> PY8 EDA -> PY9 notebook QA -> PY10 integration.
- Every lesson: learner book -> mapped source if useful -> guided cell -> changed cell -> validation -> explain-back -> saved evidence.
- Complete M-PY after PY0-PY10. Aggregate line sales to the correct grain before combining order-level returns so refunds are not multiplied.
- Gate A HelpDesk is normal. A clean pass ends M07.
- Gate B Delivery and Gate C Subscription are fresh remediation full retests only after prior failure + review + repair + explicit assignment.

Critical failures

Silent deletion/correction; disabled merge validation; multiplied refund/revenue; wrong grouped grain; chart built from unvalidated data; fabricated recommendation; hidden-state notebook; or no independent reconciliation can block a pass.

PY0 - Python, Jupyter and a reproducible analyst notebook

Practice: PY0 section in C1_M07_PYTHON_FOR_ANALYSTS_CANONICAL_PRACTICE_NOTEBOOK_RC1.ipynb

1. Why this lesson matters

Python is useful only when it makes an analysis clearer, repeatable and testable. This lesson adds one analyst-sized skill and keeps the same evidence-first reasoning you already use in Excel, SQL and Power BI.

2. What you are learning

- Understand the notebook/code/output mental model. - Install a stable environment and verify it. - Run cells in order and restart/run-all before delivery.

3. What you already need / prerequisite

No Python/Jupyter skill is assumed.

4. Picture it in your head

Think of a notebook as a transparent lab notebook: explanation, code and output sit together. Restart + Run All is like asking a new analyst to reproduce the work from page one with no memory of what you clicked earlier.

5. New words before we use them

Notebook = document with markdown/code/output cells. Kernel = Python process executing cells. Hidden state = variable/import created by an earlier manual run but not reproducible from notebook order. Environment = Python + installed packages/interpreter.

6. Watch / visual source

RECOMMENDED_VISUAL. Data Analyst Bootcamp for Beginners - Jupyter installation section | 11:05:31-11:15:33. Why use it: See Jupyter installation/open/run workflow before using the course notebook. Internal fallback: PY0 learner book + notebook setup cell fully teach the reproducibility checks. Current official companion: Project Jupyter - Installing Jupyter | Current installation options.

PY0 - Follow with me once

7. Follow with me once - fully worked teacher demonstration

A notebook mixes explanation and executable code. That is useful for analysis because another person can see not only the result, but how you obtained it. A reproducible notebook should run from the first cell to the last without depending on hidden state. A cell that works only because you ran some forgotten cell earlier is not a reliable analysis. Worked pattern: `import sys print(sys.version) import pandas as pd import matplotlib print(pd.__version__) print(matplotlib.__version__)`

8. What you should see / be able to say

The version check makes your environment visible. If the notebook later behaves differently, you have evidence of what runtime was used. Validation expectation: Save shape/key/type/count/total evidence appropriate to the lesson. A cell running without an error is not by itself validation.

9. Why it worked

The code expresses a stated business/data rule and leaves objective evidence—shape, dtype, key, row count, unmatched count, grouped reconciliation, saved output or clean rerun. A cell running without an exception is not proof of correctness.

10. Common beginner mistakes

Installing packages repeatedly without checking interpreter; cells work only because earlier manual state exists; absolute paths tied to one PC; ignoring environment/version evidence.

11. If you get stuck - recovery ladder

If a variable exists only because an earlier cell was run manually, restart the kernel and run all. If imports fail, print `sys.executable` and package versions before reinstalling anything. Concept hint -> file/column/API hint -> partial skeleton -> protected review after a saved attempt -> close review and complete a changed retry.

PY0 - Do alone, validate and master

12. Now you do a small version alone

Restart the kernel, change the output folder name, run top-to-bottom, and prove no cell depends on hidden state.

13. Validate your result

Save shape/key/type/count/total evidence appropriate to the lesson. A cell running without an error is not by itself validation.

14. Teach it back in your own words

Without reading code line-by-line, explain what the input grain was, what your code changed/calculated, what one result row/value means, and which independent evidence makes you trust it. For PY0, include the lesson-specific idea in plain language.

15. Protected review + changed fresh retry

Save the PY0 notebook attempt first. If repair is needed, open the protected PY0 review, diagnose only the weak skill, close it, then code the changed task without answer material. Keep both attempts.

16. Evidence / mastery + exact next route

Save runnable notebook cells, validation output, one error/recovery note, own-words explanation, and confidence 1-5. PASS with evidence -> continue. PARTIAL/UNCERTAIN -> targeted repair + fresh retry. FAIL -> return only to the weak lesson/competency.

PY1 - Practical Python basics: values, conditions, loops and functions

Practice: PY1 section in C1_M07_PYTHON_FOR_ANALYSTS_CANONICAL_PRACTICE_NOTEBOOK_RC1.ipynb

1. Why this lesson matters

Python is useful only when it makes an analysis clearer, repeatable and testable. This lesson adds one analyst-sized skill and keeps the same evidence-first reasoning you already use in Excel, SQL and Power BI.

2. What you are learning

- Use variables and common data types.
- Write simple conditions and loops without overengineering.
- Create small reusable functions for analyst rules.

3. What you already need / prerequisite

Prerequisite: PY0 plus the earlier analyst reasoning. The new Python/pandas operation is still introduced from first principles.

4. Picture it in your head

Variables are labelled boxes, conditions are decision gates, loops are repeated steps, and functions are reusable business rules. You only need enough Python grammar to express analysis clearly.

5. New words before we use them

Variable = name bound to a value. Type = kind of value such as int/float/string/list. Boolean = True/False. Condition = logical test. Loop = repeat block. Function = named reusable code with inputs/return.

6. Watch / visual source

RECOMMENDED_VISUAL. Data Analyst Bootcamp for Beginners - Python variables through functions | 11:15:34-12:32:21. Why use it: Visual introduction to variables, conditions, loops and functions before the analyst-sized exercises. Internal fallback: PY1 learner book + practice notebook fully teach the required Python basics.

[Open visual source](#)

PY1 - Follow with me once

7. Follow with me once - fully worked teacher demonstration

Python fundamentals matter only because they let you express business logic. A variable gives a value a name. A condition chooses between outcomes. A loop repeats a small operation. A function packages a rule so you can reuse and test it. For analyst work, clarity is more important than clever code. Worked pattern: revenue = 125000 cost = 82000 profit = revenue - cost
`def sales_band(net_sales): if net_sales >= 100000: return "High" elif net_sales >= 50000: return "Medium" return "Low"`
`print(profit, sales_band(revenue))`

8. What you should see / be able to say

The business threshold lives in one function, making it easier to test boundary values such as exactly 50,000 and 100,000. Validation expectation: Save shape/key/type/count/total evidence appropriate to the lesson. A cell running without an error is not by itself validation.

9. Why it worked

The code expresses a stated business/data rule and leaves objective evidence—shape, dtype, key, row count, unmatched count, grouped reconciliation, saved output or clean rerun. A cell running without an exception is not proof of correctness.

10. Common beginner mistakes

Changing thresholds without boundary tests; using a loop when a simple expression is clearer; function silently depends on a global variable; clever one-liner nobody can explain.

11. If you get stuck - recovery ladder

If a condition fails at a boundary, test values just below, exactly at, and just above the threshold. If a loop is confusing, print the current item and accumulator each iteration. Concept hint -> file/column/API hint -> partial skeleton -> protected review after a saved attempt -> close review and complete a changed retry.

PY1 - Do alone, validate and master

12. Now you do a small version alone

Write a changed business rule with exact boundary tests, then process a list with a for-loop and a function without copying the guided example.

13. Validate your result

Save shape/key/type/count/total evidence appropriate to the lesson. A cell running without an error is not by itself validation.

14. Teach it back in your own words

Without reading code line-by-line, explain what the input grain was, what your code changed/calculated, what one result row/value means, and which independent evidence makes you trust it. For PY1, include the lesson-specific idea in plain language.

15. Protected review + changed fresh retry

Save the PY1 notebook attempt first. If repair is needed, open the protected PY1 review, diagnose only the weak skill, close it, then code the changed task without answer material. Keep both attempts.

16. Evidence / mastery + exact next route

Save runnable notebook cells, validation output, one error/recovery note, own-words explanation, and confidence 1-5. PASS with evidence -> continue. PARTIAL/UNCERTAIN -> targeted repair + fresh retry. FAIL -> return only to the weak lesson/competency.

PY2 - pandas: load, inspect, select and filter

Practice: PY2 section in C1_M07_PYTHON_FOR_ANALYSTS_CANONICAL_PRACTICE_NOTEBOOK_RC1.ipynb

1. Why this lesson matters

Python is useful only when it makes an analysis clearer, repeatable and testable. This lesson adds one analyst-sized skill and keeps the same evidence-first reasoning you already use in Excel, SQL and Power BI.

2. What you are learning

- Load CSV/Excel/JSON data. - Inspect shape, columns, types, missingness and samples. - Select columns and filter rows using explicit conditions.

3. What you already need / prerequisite

Prerequisite: PY1 plus the earlier analyst reasoning. The new Python/pandas operation is still introduced from first principles.

4. Picture it in your head

A DataFrame is a table in memory. Loading is opening the file; inspection is checking what actually arrived before you trust column names, types, missing values or row counts.

5. New words before we use them

DataFrame = labelled 2D table. Series = one labelled column-like object. dtype = stored type. `shape` = rows/columns. `.loc[]` = label/condition-based selection. Missing value = absent/unknown representation.

6. Watch / visual source

RECOMMENDED_VISUAL. Alex The Analyst - Reading in Files in Pandas | 0:58-18:05. Why use it: See CSV/JSON/Excel loading and DataFrame inspection in a focused beginner pandas lesson. Internal fallback: PY2 learner book + practice notebook + current pandas read_csv reference. Current official companion: pandas - read_csv | filepath, dtype, parse/read behavior.

[Open visual source](#)

[Open official reference](#)

PY2 - Follow with me once

7. Follow with me once - fully worked teacher demonstration

A pandas DataFrame is a table-like object. Before analysis, inspect the dataset rather than assuming its shape or data types. Useful first checks: `head()`, `shape`, `columns`, `dtypes`, `info()`, missing counts and key uniqueness. Filtering changes which rows are in scope, so filters must reflect the business question. Worked pattern: `import pandas as pd sales = pd.read_csv("Practice_Data/sales.csv") print(sales.head()) print(sales.shape) print(sales.dtypes) print(sales.isna().sum()) east = sales.loc[sales["region"].eq("East"), ["order_id", "order_date", "product_id", "quantity"]]`

8. What you should see / be able to say

Inspection comes before analysis. ``shape`` and missing counts tell you what you actually loaded; ``loc[]`` makes filter intent explicit. Validation expectation: Save shape/key/type/count/total evidence appropriate to the lesson. A cell running without an error is not by itself validation.

9. Why it worked

The code expresses a stated business/data rule and leaves objective evidence—shape, dtype, key, row count, unmatched count, grouped reconciliation, saved output or clean rerun. A cell running without an exception is not proof of correctness.

10. Common beginner mistakes

Analyzing before ``shape`/dtypes/missing/key` checks; date comparison on text; chained confusing filters; assuming a file loaded because no visible exception occurred.

11. If you get stuck - recovery ladder

If filtering returns unexpected rows, inspect dtype and unique values before changing the condition. If a date comparison fails, parse the column explicitly first. Concept hint -> file/column/API hint -> partial skeleton -> protected review after a saved attempt -> close review and complete a changed retry.

PY2 - Do alone, validate and master

12. Now you do a small version alone

Load a fresh CSV, report shape/types/missing/key uniqueness, then filter two conditions and explain the business scope.

13. Validate your result

Save shape/key/type/count/total evidence appropriate to the lesson. A cell running without an error is not by itself validation.

14. Teach it back in your own words

Without reading code line-by-line, explain what the input grain was, what your code changed/calculated, what one result row/value means, and which independent evidence makes you trust it. For PY2, include the lesson-specific idea in plain language.

15. Protected review + changed fresh retry

Save the PY2 notebook attempt first. If repair is needed, open the protected PY2 review, diagnose only the weak skill, close it, then code the changed task without answer material. Keep both attempts.

16. Evidence / mastery + exact next route

Save runnable notebook cells, validation output, one error/recovery note, own-words explanation, and confidence 1-5. PASS with evidence -> continue. PARTIAL/UNCERTAIN -> targeted repair + fresh retry. FAIL -> return only to the weak lesson/competency.

PY3 - Data quality and cleaning in pandas

Practice: PY3 section in C1_M07_PYTHON_FOR_ANALYSTS_CANONICAL_PRACTICE_NOTEBOOK_RC1.ipynb

1. Why this lesson matters

Python is useful only when it makes an analysis clearer, repeatable and testable. This lesson adds one analyst-sized skill and keeps the same evidence-first reasoning you already use in Excel, SQL and Power BI.

2. What you are learning

- Separate detection from correction.
- Clean text and dates transparently.
- Treat duplicates, missing data and invalid values using business rules.

3. What you already need / prerequisite

Prerequisite: PY2 plus the earlier analyst reasoning. The new Python/pandas operation is still introduced from first principles.

4. Picture it in your head

Cleaning is an evidence-preserving repair bench: first mark the problem, then decide whether/how to repair it. Never sweep a suspicious row away merely because a warning disappears.

5. New words before we use them

Detection = identify issue without changing it. Correction = justified change. ``duplicated()`` = duplicate flag. ``pd.to_datetime(..., errors='coerce')`` = invalid parse becomes missing/NaT. Issue log = evidence of problem, decision and impact.

6. Watch / visual source

RECOMMENDED_VISUAL. Data Analyst Bootcamp for Beginners - Data Cleaning in Pandas | 15:33:02-16:11:38. Why use it: See common cleaning operations, then return to the course's stricter detection-before-correction workflow. Internal fallback: PY3 learner book + practice data quality checklist fully teach the controlled cleaning route.

[Open visual source](#)

PY3 - Follow with me once

7. Follow with me once - fully worked teacher demonstration

Cleaning is not 'make warnings disappear.' Preserve evidence, identify the rule, investigate impact and document the decision. Use `.str.strip()` for whitespace, `pd.to_datetime(..., errors='coerce')` when you want invalid date text to become visible as missing, `duplicated()` for duplicate detection, and explicit range/rule checks for invalid values. Worked pattern: `work = sales.copy()`
`work["customer_id"] = work["customer_id"].astype("string").str.strip()` `work["region"] =`
`work["region"].astype("string").str.strip().str.title()` `work["order_date"] = pd.to_datetime(work["order_date"], errors="coerce")`
`duplicate_lines = work[work.duplicated("order_line_id", keep=False)]` `bad_discount = work.loc[~work["discount"].between(0,1)]`

8. What you should see / be able to say

``errors='coerce'`` does not fix a bad date. It exposes values that could not be parsed so you can investigate them. Validation expectation: Save shape/key/type/count/total evidence appropriate to the lesson. A cell running without an error is not by itself validation.

9. Why it worked

The code expresses a stated business/data rule and leaves objective evidence—shape, dtype, key, row count, unmatched count, grouped reconciliation, saved output or clean rerun. A cell running without an exception is not proof of correctness.

10. Common beginner mistakes

Blanket ``dropna()``/``drop_duplicates()``; ``fillna(0)`` changing business meaning; overwriting raw DataFrame; coercing invalid dates then treating them as fixed; deleting both/one duplicate without stating the key/grain rule.

11. If you get stuck - recovery ladder

If cleaning reduces row count unexpectedly, compare before/after counts and isolate the exact rule. Never use `dropna/drop_duplicates` until you can name the business reason. Concept hint -> file/column/API hint -> partial skeleton -> protected review after a saved attempt -> close review and complete a changed retry.

PY3 - Do alone, validate and master

12. Now you do a small version alone

On a changed dirty sample, separate detection from correction; keep an issue log and prove no blanket fill/drop hid evidence.

13. Validate your result

Save shape/key/type/count/total evidence appropriate to the lesson. A cell running without an error is not by itself validation.

14. Teach it back in your own words

Without reading code line-by-line, explain what the input grain was, what your code changed/calculated, what one result row/value means, and which independent evidence makes you trust it. For PY3, include the lesson-specific idea in plain language.

15. Protected review + changed fresh retry

Save the PY3 notebook attempt first. If repair is needed, open the protected PY3 review, diagnose only the weak skill, close it, then code the changed task without answer material. Keep both attempts.

16. Evidence / mastery + exact next route

Save runnable notebook cells, validation output, one error/recovery note, own-words explanation, and confidence 1-5. PASS with evidence -> continue. PARTIAL/UNCERTAIN -> targeted repair + fresh retry. FAIL -> return only to the weak lesson/competency.

PY4 - GroupBy, aggregation and result grain

Practice: PY4 section in C1_M07_PYTHON_FOR_ANALYSTS_CANONICAL_PRACTICE_NOTEBOOK_RC1.ipynb

1. Why this lesson matters

Python is useful only when it makes an analysis clearer, repeatable and testable. This lesson adds one analyst-sized skill and keeps the same evidence-first reasoning you already use in Excel, SQL and Power BI.

2. What you are learning

- Aggregate measures by meaningful business dimensions. - Use named aggregation for readable outputs. - State what one result row represents.

3. What you already need / prerequisite

Prerequisite: PY3 plus the earlier analyst reasoning. The new Python/pandas operation is still introduced from first principles.

4. Picture it in your head

GroupBy is a bucket machine. You choose bucket labels such as Region, then calculate one summary row per bucket. The output grain changes, so row count and business meaning change too.

5. New words before we use them

GroupBy = split rows into groups. Aggregation = reduce many rows to summary value. Named aggregation = explicit output name + input + function. `nunique` = distinct count. Result grain = what one output row represents.

6. Watch / visual source

RECOMMENDED_VISUAL. Data Analyst Bootcamp for Beginners - Group By and Aggregate Functions | 14:42:59-14:54:03. Why use it: See groupby/aggregation syntax before the course makes result grain explicit. Internal fallback: PY4 learner book + practice notebook fully teach named aggregation and reconciliation.

[Open visual source](#)

PY4 - Follow with me once

7. Follow with me once - fully worked teacher demonstration

groupby changes grain. If you group by region, one output row represents one region. If you group by month and category, each output row represents one month-category combination. Always check whether a metric is additive at that grain. Counts, distinct counts and averages answer different questions. Worked pattern: `summary = ( work.groupby("region", dropna=False) .agg( rows=("order_line_id", "size"), orders=("order_id", "nunique"), net_sales=("net_sales", "sum") ) .reset_index() )`

8. What you should see / be able to say

`rows` and `orders` are different metrics. Grouping changes result grain, so you must know whether you are counting lines or distinct orders. Validation expectation: Save shape/key/type/count/total evidence appropriate to the lesson. A cell running without an error is not by itself validation.

9. Why it worked

The code expresses a stated business/data rule and leaves objective evidence—shape, dtype, key, row count, unmatched count, grouped reconciliation, saved output or clean rerun. A cell running without an exception is not proof of correctness.

10. Common beginner mistakes

Confusing row count with distinct orders; grouping by wrong columns; averaging already-aggregated numbers carelessly; not reconciling additive totals after grouping.

11. If you get stuck - recovery ladder

If totals look wrong, state input grain and output grain, then reconcile the grouped sum to the ungrouped total. Distinguish row count from nunique business entities. Concept hint -> file/column/API hint -> partial skeleton -> protected review after a saved attempt -> close review and complete a changed retry.

PY4 - Do alone, validate and master

12. Now you do a small version alone

Create a changed grouped summary with rows, distinct entities, sum and mean; state exactly what one output row represents.

13. Validate your result

Save shape/key/type/count/total evidence appropriate to the lesson. A cell running without an error is not by itself validation.

14. Teach it back in your own words

Without reading code line-by-line, explain what the input grain was, what your code changed/calculated, what one result row/value means, and which independent evidence makes you trust it. For PY4, include the lesson-specific idea in plain language.

15. Protected review + changed fresh retry

Save the PY4 notebook attempt first. If repair is needed, open the protected PY4 review, diagnose only the weak skill, close it, then code the changed task without answer material. Keep both attempts.

16. Evidence / mastery + exact next route

Save runnable notebook cells, validation output, one error/recovery note, own-words explanation, and confidence 1-5. PASS with evidence -> continue. PARTIAL/UNCERTAIN -> targeted repair + fresh retry. FAIL -> return only to the weak lesson/competency.

PY5 - Merge, concat and validation

Practice: PY5 section in C1_M07_PYTHON_FOR_ANALYSTS_CANONICAL_PRACTICE_NOTEBOOK_RC1.ipynb

1. Why this lesson matters

Python is useful only when it makes an analysis clearer, repeatable and testable. This lesson adds one analyst-sized skill and keeps the same evidence-first reasoning you already use in Excel, SQL and Power BI.

2. What you are learning

- Choose join keys intentionally.
- Detect one-to-many risks before merging.
- Validate row counts and unmatched keys after merge.

3. What you already need / prerequisite

Prerequisite: PY4 plus the earlier analyst reasoning. The new Python/pandas operation is still introduced from first principles.

4. Picture it in your head

A merge is a controlled bridge between tables. ``validate=`` checks whether the bridge shape matches your assumption; ``indicator=True`` shows which rows crossed and which did not.

5. New words before we use them

Merge = combine DataFrames on keys. ``validate='many_to_one'`` = assert right key uniqueness for fact-to-dimension enrichment. ``indicator=True`` = create ``_merge`` source-status column. Unmatched = key absent on other side. Concat = stack/align objects.

6. Watch / visual source

RECOMMENDED_VISUAL. Data Analyst Bootcamp for Beginners - Merging DataFrames | 14:54:04-15:16:12. Why use it: See merge syntax before using `validate=` and `indicator=True` as correctness controls. Internal fallback: PY5 learner book + current pandas merge docs + practice notebook. Current official companion: pandas - merge | how, indicator and validate parameters.

[Open visual source](#)

[Open official reference](#)

PY5 - Follow with me once

7. Follow with me once - fully worked teacher demonstration

A merge is the Python equivalent of a relational join. It can silently multiply rows if the right-side key is not unique. Before a many-to-one enrichment merge, test the dimension key for duplicates. During merge, `validate='many\_to\_one'` makes pandas raise an error if the assumed relationship is false. `indicator=True` exposes unmatched rows. Worked pattern: `products = pd.read_csv("Practice_Data/products.csv") assert not products["product_id"].duplicated().any() merged = work.merge( products, on="product_id", how="left", validate="many_to_one", indicator=True ) print(merged["_merge"].value_counts()) print(len(work), len(merged))`

8. What you should see / be able to say

`validate='many\_to\_one'` is a correctness control. `\_merge` tells you which transaction keys did not match the master. Validation expectation: Save shape/key/type/count/total evidence appropriate to the lesson. A cell running without an error is not by itself validation.

9. Why it worked

The code expresses a stated business/data rule and leaves objective evidence—shape, dtype, key, row count, unmatched count, grouped reconciliation, saved output or clean rerun. A cell running without an exception is not proof of correctness.

10. Common beginner mistakes

Removing `validate=` when it raises; cleaning key values after merge instead of before; duplicate dimension key silently multiplying fact rows; dropping `left\_only` without investigation; pandas null-key matching assumed identical to SQL behavior.

11. If you get stuck - recovery ladder

If merge validation fails, stop and inspect duplicate keys on each side. Do not remove `validate=` to make the error disappear. Concept hint -> file/column/API hint -> partial skeleton -> protected review after a saved attempt -> close review and complete a changed retry.

PY5 - Do alone, validate and master

12. Now you do a small version alone

Merge a changed fact/dimension pair using `validate=` and `indicator=True`; prove key uniqueness and explain any unmatched rows.

13. Validate your result

Save shape/key/type/count/total evidence appropriate to the lesson. A cell running without an error is not by itself validation.

14. Teach it back in your own words

Without reading code line-by-line, explain what the input grain was, what your code changed/calculated, what one result row/value means, and which independent evidence makes you trust it. For PY5, include the lesson-specific idea in plain language.

15. Protected review + changed fresh retry

Save the PY5 notebook attempt first. If repair is needed, open the protected PY5 review, diagnose only the weak skill, close it, then code the changed task without answer material. Keep both attempts.

16. Evidence / mastery + exact next route

Save runnable notebook cells, validation output, one error/recovery note, own-words explanation, and confidence 1-5. PASS with evidence -> continue. PARTIAL/UNCERTAIN -> targeted repair + fresh retry. FAIL -> return only to the weak lesson/competency.

PY6 - Reshape and time analysis

Practice: PY6 section in C1_M07_PYTHON_FOR_ANALYSTS_CANONICAL_PRACTICE_NOTEBOOK_RC1.ipynb

1. Why this lesson matters

Python is useful only when it makes an analysis clearer, repeatable and testable. This lesson adds one analyst-sized skill and keeps the same evidence-first reasoning you already use in Excel, SQL and Power BI.

2. What you are learning

- Parse dates and create time attributes.
- Pivot long data to a reporting matrix.
- Return wide summaries to long/tidy form when useful.

3. What you already need / prerequisite

Prerequisite: PY5 plus the earlier analyst reasoning. The new Python/pandas operation is still introduced from first principles.

4. Picture it in your head

Long data stacks observations down rows; wide data spreads categories across columns. Pivot and melt change shape, not underlying business truth, so additive totals should reconcile.

5. New words before we use them

Datetime = parsed date/time value. Month key = chronological month representation. Pivot = long-to-wide summary. Melt = wide-to-long reshape. Tidy/long = one observation per row with variables in columns.

6. Watch / visual source

RECOMMENDED_VISUAL. Data Analyst Bootcamp for Beginners - indexes/aggregation/merge context | 14:31:37-15:16:12. Why use it: Provides visual context for indexes/grouping/merging before the course teaches tidy monthly reshape. Internal fallback: PY6 learner book + practice notebook teach datetime/pivot/melt and reconciliation explicitly.

[Open visual source](#)

PY6 - Follow with me once

7. Follow with me once - fully worked teacher demonstration

Analyst data often moves between long and wide forms. Long form is usually easier for filtering, grouping and plotting. A pivot table is useful for a matrix view such as Month x Region. Time analysis begins with real datetime values, not strings that merely look like dates. Worked pattern: `merged["order_date"] = pd.to_datetime(merged["order_date"], errors="coerce")` `merged["month"] = merged["order_date"].dt.to_period("M").astype(str)` `monthly = merged.groupby(["month", "region"], as_index=False)["net_sales"].sum()` `wide = monthly.pivot(index="month", columns="region", values="net_sales")` `long_again = wide.reset_index().melt(id_vars="month", var_name="region", value_name="net_sales")`

8. What you should see / be able to say

Do not build month logic from text slicing when you already have datetime values. Date types let you derive periods consistently. Validation expectation: Save shape/key/type/count/total evidence appropriate to the lesson. A cell running without an error is not by itself validation.

9. Why it worked

The code expresses a stated business/data rule and leaves objective evidence—shape, dtype, key, row count, unmatched count, grouped reconciliation, saved output or clean rerun. A cell running without an exception is not proof of correctness.

10. Common beginner mistakes

Using month-name text as chronological order; pivoting before defining aggregation; totals change after reshape without investigation; duplicate index/column combinations misunderstood.

11. If you get stuck - recovery ladder

If pivot/melt totals change, compare total before reshape and after returning to long form. Check duplicate index/column combinations. Concept hint -> file/column/API hint -> partial skeleton -> protected review after a saved attempt -> close review and complete a changed retry.

PY6 - Do alone, validate and master

12. Now you do a small version alone

Build a changed monthly-region table, pivot wide, then return to long form; prove row/total reconciliation.

13. Validate your result

Save shape/key/type/count/total evidence appropriate to the lesson. A cell running without an error is not by itself validation.

14. Teach it back in your own words

Without reading code line-by-line, explain what the input grain was, what your code changed/calculated, what one result row/value means, and which independent evidence makes you trust it. For PY6, include the lesson-specific idea in plain language.

15. Protected review + changed fresh retry

Save the PY6 notebook attempt first. If repair is needed, open the protected PY6 review, diagnose only the weak skill, close it, then code the changed task without answer material. Keep both attempts.

16. Evidence / mastery + exact next route

Save runnable notebook cells, validation output, one error/recovery note, own-words explanation, and confidence 1-5. PASS with evidence -> continue. PARTIAL/UNCERTAIN -> targeted repair + fresh retry. FAIL -> return only to the weak lesson/competency.

PY7 - Visualization with pandas and Matplotlib

Practice: PY7 section in C1_M07_PYTHON_FOR_ANALYSTS_CANONICAL_PRACTICE_NOTEBOOK_RC1.ipynb

1. Why this lesson matters

Python is useful only when it makes an analysis clearer, repeatable and testable. This lesson adds one analyst-sized skill and keeps the same evidence-first reasoning you already use in Excel, SQL and Power BI.

2. What you are learning

- Choose charts based on analytical question. - Create readable line/bar/histogram plots. - Label axes/titles and avoid decorative complexity.

3. What you already need / prerequisite

Prerequisite: PY6 plus the earlier analyst reasoning. The new Python/pandas operation is still introduced from first principles.

4. Picture it in your head

A chart is a visual answer to a question: time moves left-to-right, categories compare lengths, distributions show spread. Summarize correctly before decorating.

5. New words before we use them

Figure = complete Matplotlib canvas. Axes = plotting area. Line chart = ordered trend. Bar chart = category comparison. Histogram = numeric distribution. Label = title/axis text communicating meaning.

6. Watch / visual source

RECOMMENDED_VISUAL. Data Analyst Bootcamp for Beginners - Creating Visualizations using pandas | 15:16:13-15:33:01. Why use it: See pandas/Matplotlib chart creation before selecting charts by business question. Internal fallback: PY7 learner book + current Matplotlib plot-types reference. Current official companion: Matplotlib - Plot types | line/bar/histogram overview.

[Open visual source](#)

[Open official reference](#)

PY7 - Follow with me once

7. Follow with me once - fully worked teacher demonstration

Use a line chart for ordered time trends, a bar chart for category comparison and a histogram/boxplot for distribution questions. A chart is part of the argument, not decoration. The goal is not to memorize every Matplotlib option. Create a clear first chart, then improve labels and formatting only as needed. Worked pattern: `import matplotlib.pyplot as plt trend = merged.groupby("month", as_index=False)["net_sales"].sum() ax = trend.plot(x="month", y="net_sales", kind="line", marker="o", legend=False)`
`ax.set_title("Monthly Net Sales") ax.set_xlabel("Month") ax.set_ylabel("Net Sales") plt.xticks(rotation=45) plt.tight_layout()`
`plt.show()`

8. What you should see / be able to say

The plot answers a defined question. Keep chart code simple enough that you can explain every line. Validation expectation: Save shape/key/type/count/total evidence appropriate to the lesson. A cell running without an error is not by itself validation.

9. Why it worked

The code expresses a stated business/data rule and leaves objective evidence—shape, dtype, key, row count, unmatched count, grouped reconciliation, saved output or clean rerun. A cell running without an exception is not proof of correctness.

10. Common beginner mistakes

Plotting raw line rows when the question needs monthly summary; multiple unrelated charts in one figure; unreadable labels; decorative 3D; formatting before validating source aggregation.

11. If you get stuck - recovery ladder

If the chart looks wrong, inspect the summarized data feeding the chart before formatting. Chart polish cannot repair wrong aggregation. Concept hint -> file/column/API hint -> partial skeleton -> protected review after a saved attempt -> close review and complete a changed retry.

PY7 - Do alone, validate and master

12. Now you do a small version alone

Create one time trend and one category comparison on changed data; justify chart type and label axes clearly.

13. Validate your result

Save shape/key/type/count/total evidence appropriate to the lesson. A cell running without an error is not by itself validation.

14. Teach it back in your own words

Without reading code line-by-line, explain what the input grain was, what your code changed/calculated, what one result row/value means, and which independent evidence makes you trust it. For PY7, include the lesson-specific idea in plain language.

15. Protected review + changed fresh retry

Save the PY7 notebook attempt first. If repair is needed, open the protected PY7 review, diagnose only the weak skill, close it, then code the changed task without answer material. Keep both attempts.

16. Evidence / mastery + exact next route

Save runnable notebook cells, validation output, one error/recovery note, own-words explanation, and confidence 1-5. PASS with evidence -> continue. PARTIAL/UNCERTAIN -> targeted repair + fresh retry. FAIL -> return only to the weak lesson/competency.

PY8 - Exploratory data analysis (EDA)

Practice: PY8 section in C1_M07_PYTHON_FOR_ANALYSTS_CANONICAL_PRACTICE_NOTEBOOK_RC1.ipynb

1. Why this lesson matters

Python is useful only when it makes an analysis clearer, repeatable and testable. This lesson adds one analyst-sized skill and keeps the same evidence-first reasoning you already use in Excel, SQL and Power BI.

2. What you are learning

- Perform a structured first-pass analysis.
- Separate suspicious data from business findings.
- Use grouped analysis and visuals to form evidence-based observations.

3. What you already need / prerequisite

Prerequisite: PY7 plus the earlier analyst reasoning. The new Python/pandas operation is still introduced from first principles.

4. Picture it in your head

EDA is a guided investigation: structure -> quality -> distributions -> comparisons -> trends -> suspicious values -> findings -> limitations. It is not a contest to make the most charts.

5. New words before we use them

EDA = exploratory data analysis. Distribution = how values are spread. Outlier = unusual value requiring context. Correlation = association measure, not proof of causation. Finding = supported observation. Limitation = what evidence cannot prove.

6. Watch / visual source

RECOMMENDED_VISUAL. Alex The Analyst - Exploratory Data Analysis in Pandas | 1:51-30:41. Why use it: See a compact EDA flow from first look through grouped analysis and visual exploration; Seaborn heatmap is optional, not required. Internal fallback: PY8 learner book + practice notebook define the course EDA contract.

[Open visual source](#)

PY8 - Follow with me once

7. Follow with me once - fully worked teacher demonstration

EDA is not 'make every chart.' Start with structure and quality, then examine distributions, categories, trends, grouped metrics and unusual values that matter to the question. An observation becomes useful when you connect it to context: what changed, where, how much, compared with what, and what limitation remains. Worked pattern: `print(merged.shape)` `print(merged.info())` `print(merged.describe(include="all"))` `print(merged.isna().sum())` # Then: category counts, grouped metrics, trends, # selected plots, findings and limitations.

8. What you should see / be able to say

Correlation or a striking chart is not automatic causation. EDA produces questions and evidence, not magical explanations. Validation expectation: Save shape/key/type/count/total evidence appropriate to the lesson. A cell running without an error is not by itself validation.

9. Why it worked

The code expresses a stated business/data rule and leaves objective evidence—shape, dtype, key, row count, unmatched count, grouped reconciliation, saved output or clean rerun. A cell running without an exception is not proof of correctness.

10. Common beginner mistakes

Chart dump; data-quality issue presented as business behavior; correlation claimed as cause; suspicious values silently removed; findings have no comparison/magnitude/limitation.

11. If you get stuck - recovery ladder

If EDA becomes a chart dump, return to the business question and keep only checks/visuals that reveal structure, quality, comparison or trend. Concept hint -> file/column/API hint -> partial skeleton -> protected review after a saved attempt -> close review and complete a changed retry.

PY8 - Do alone, validate and master

12. Now you do a small version alone

Run a compact EDA on a changed dataset: structure, quality, distributions, groups, trends, suspicious values, findings and limitations.

13. Validate your result

Save shape/key/type/count/total evidence appropriate to the lesson. A cell running without an error is not by itself validation.

14. Teach it back in your own words

Without reading code line-by-line, explain what the input grain was, what your code changed/calculated, what one result row/value means, and which independent evidence makes you trust it. For PY8, include the lesson-specific idea in plain language.

15. Protected review + changed fresh retry

Save the PY8 notebook attempt first. If repair is needed, open the protected PY8 review, diagnose only the weak skill, close it, then code the changed task without answer material. Keep both attempts.

16. Evidence / mastery + exact next route

Save runnable notebook cells, validation output, one error/recovery note, own-words explanation, and confidence 1-5. PASS with evidence -> continue. PARTIAL/UNCERTAIN -> targeted repair + fresh retry. FAIL -> return only to the weak lesson/competency.

PY9 - Notebook quality, outputs and reproducibility

Practice: PY9 section in C1_M07_PYTHON_FOR_ANALYSTS_CANONICAL_PRACTICE_NOTEBOOK_RC1.ipynb

1. Why this lesson matters

Python is useful only when it makes an analysis clearer, repeatable and testable. This lesson adds one analyst-sized skill and keeps the same evidence-first reasoning you already use in Excel, SQL and Power BI.

2. What you are learning

- Organize a notebook so another analyst can follow it. - Save important outputs intentionally. - Run top-to-bottom and avoid hidden state.

3. What you already need / prerequisite

Prerequisite: PY8 plus the earlier analyst reasoning. The new Python/pandas operation is still introduced from first principles.

4. Picture it in your head

A professional notebook is a recipe another person can cook from top to bottom: setup -> ingredients/data -> checks -> preparation -> analysis -> validation -> output. Hidden state is a missing recipe step.

5. New words before we use them

Reproducibility = work can be rerun from inputs/code. Relative path = path based on project folder, not one person's machine. Output artifact = saved table/chart/file. Restart + Run All = execute notebook in order from clean kernel.

6. Watch / visual source

BOOK_LED. Internal lesson - notebook quality and reproducibility. Why use it: The skill is best proven by Restart + Run All, file paths, ordered sections and saved outputs in the learner's own notebook. Internal fallback: PY9 learner book + practice notebook. Current official companion: pandas - DataFrame.to_csv | path/index/output parameters.

[Open official reference](#)

PY9 - Follow with me once

7. Follow with me once - fully worked teacher demonstration

A professional notebook should have a visible question, imports/setup, loading, quality checks, preparation, analysis, validation, visuals, findings and limitations. Do not scatter unexplained code across random cells. Avoid writing over raw data. Save clean outputs with clear filenames. Before delivery: Restart Kernel -> Run All -> confirm no error. Worked pattern: `from pathlib import Path OUTPUT = Path("outputs") OUTPUT.mkdir(exist_ok=True) analysis_table.to_csv(OUTPUT / "analysis_table.csv", index=False)`

8. What you should see / be able to say

Running from top to bottom is a QA test. It catches hidden-state errors and missing setup cells. Validation expectation: Save shape/key/type/count/total evidence appropriate to the lesson. A cell running without an error is not by itself validation.

9. Why it worked

The code expresses a stated business/data rule and leaves objective evidence—shape, dtype, key, row count, unmatched count, grouped reconciliation, saved output or clean rerun. A cell running without an exception is not proof of correctness.

10. Common beginner mistakes

Notebook only works after random cell order; output directory assumed to exist; overwriting raw/important result; absolute user-specific paths; no final clean run.

11. If you get stuck - recovery ladder

If Restart + Run All fails, the notebook has hidden state, missing setup, or path assumptions. Fix the earliest failing cell, not the last visible symptom. Concept hint -> file/column/API hint -> partial skeleton -> protected review after a saved attempt -> close review and complete a changed retry.

PY9 - Do alone, validate and master

12. Now you do a small version alone

Take a deliberately out-of-order notebook, reorganize it into setup/load/quality/prepare/analyze/validate/output, then Restart + Run All.

13. Validate your result

Save shape/key/type/count/total evidence appropriate to the lesson. A cell running without an error is not by itself validation.

14. Teach it back in your own words

Without reading code line-by-line, explain what the input grain was, what your code changed/calculated, what one result row/value means, and which independent evidence makes you trust it. For PY9, include the lesson-specific idea in plain language.

15. Protected review + changed fresh retry

Save the PY9 notebook attempt first. If repair is needed, open the protected PY9 review, diagnose only the weak skill, close it, then code the changed task without answer material. Keep both attempts.

16. Evidence / mastery + exact next route

Save runnable notebook cells, validation output, one error/recovery note, own-words explanation, and confidence 1-5. PASS with evidence -> continue. PARTIAL/UNCERTAIN -> targeted repair + fresh retry. FAIL -> return only to the weak lesson/competency.

PY10 - Integrated analyst workflow in Python

Practice: PY10 section in C1_M07_PYTHON_FOR_ANALYSTS_CANONICAL_PRACTICE_NOTEBOOK_RC1.ipynb

1. Why this lesson matters

Python is useful only when it makes an analysis clearer, repeatable and testable. This lesson adds one analyst-sized skill and keeps the same evidence-first reasoning you already use in Excel, SQL and Power BI.

2. What you are learning

- Combine framing, quality, cleaning, merging, analysis and communication.
- Choose the smallest useful set of techniques.
- Defend your result with validation evidence.

3. What you already need / prerequisite

Prerequisite: PY9 plus the earlier analyst reasoning. The new Python/pandas operation is still introduced from first principles.

4. Picture it in your head

The final skill is orchestration: choose only the tools the question needs, preserve evidence, validate joins/totals, communicate the answer and prove the notebook reruns cleanly.

5. New words before we use them

Workflow = ordered analysis stages. Metric definition = exact business/calculation meaning. Reconciliation = independent agreement of totals/counts. Recommendation = action supported by evidence. Hypothesis = possible explanation not yet proven.

6. Watch / visual source

BOOK_LED. Internal lesson - integrated analyst workflow. Why use it: No new library is introduced; this lesson assesses integration of already taught skills. Internal fallback: PY10 learner book + integrated M-PY route.

PY10 - Follow with me once

7. Follow with me once - fully worked teacher demonstration

The final Python skill is not a new function. It is integration. You receive an unfamiliar request and unfamiliar data, then decide what to check, what to calculate, how to validate and how to communicate. Your notebook is evidence of reasoning. A complicated notebook with weak validation is worse than a simple notebook that answers the business question correctly. Worked pattern: # Final workflow # 1. Frame question/metrics # 2. Load/inspect # 3. Quality audit # 4. Clean/prepare # 5. Merge + validate # 6. Analyze/group/reshape # 7. Visualize # 8. Validate results # 9. Findings/recommendation/limitations # 10. Restart + Run All + save outputs

8. What you should see / be able to say

Tool choice is subordinate to the question. The Gate rewards correct reasoning, validation and communication, not maximum code. Validation expectation: Save shape/key/type/count/total evidence appropriate to the lesson. A cell running without an error is not by itself validation.

9. Why it worked

The code expresses a stated business/data rule and leaves objective evidence—shape, dtype, key, row count, unmatched count, grouped reconciliation, saved output or clean rerun. A cell running without an exception is not proof of correctness.

10. Common beginner mistakes

Using every technique instead of the smallest useful set; returns merged to line rows and refund multiplied; recommendation stronger than evidence; no independent reconciliation; delivery notebook cannot rerun.

11. If you get stuck - recovery ladder

If the project becomes too complicated, reduce it to one question, a small metric set, explicit validation, and evidence-based findings. More code is not a better analysis. Concept hint -> file/column/API hint -> partial skeleton -> protected review after a saved attempt -> close review and complete a changed retry.

PY10 - Do alone, validate and master

12. Now you do a small version alone

Complete a fresh mini-case end-to-end with only the business request and files: frame -> audit -> prepare -> merge -> analyze -> visualize -> validate -> communicate -> rerun.

13. Validate your result

Save shape/key/type/count/total evidence appropriate to the lesson. A cell running without an error is not by itself validation.

14. Teach it back in your own words

Without reading code line-by-line, explain what the input grain was, what your code changed/calculated, what one result row/value means, and which independent evidence makes you trust it. For PY10, include the lesson-specific idea in plain language.

15. Protected review + changed fresh retry

Save the PY10 notebook attempt first. If repair is needed, open the protected PY10 review, diagnose only the weak skill, close it, then code the changed task without answer material. Keep both attempts.

16. Evidence / mastery + exact next route

Save runnable notebook cells, validation output, one error/recovery note, own-words explanation, and confidence 1-5. PASS with evidence -> continue. PARTIAL/UNCERTAIN -> targeted repair + fresh retry. FAIL -> return only to the weak lesson/competency.

M-PY - Integrated analyst workflow project

1. Why this project matters

An employer will hand you files and a question, not a list of pandas methods. The project proves you can choose a small useful workflow, protect data quality, validate merges/totals, communicate findings and deliver a reproducible notebook.

2. What you are building

Sales/returns/delivery analysis from Practice_Data with an auditable notebook, saved output table(s), separate useful figures, three findings, one recommendation, limitations and a clean Restart + Run All.

3. Prerequisite

PY0-PY10 fresh evidence. Use only techniques already taught. The project does not require web scraping, machine learning, Seaborn or clever advanced Python.

4. Picture it in your head

Question -> raw frames -> issue log -> strict analysis set -> validated dimensions/merge -> order/region/month summaries -> returns at safe grain -> visuals -> reconciliations -> findings -> outputs -> clean rerun.

5. New words before we use them

Strict analysis set = rows accepted by documented rules. Refund grain = one return record/order here. Analysis table = saved result table. Reproducible = reruns from source files. Limitation = what the data/process does not justify claiming.

6. Watch / visual source

INTERNAL PROJECT TEACHER. No new external source is required. Rewatch only a named weak PY lesson; the project should prove integration rather than video following.

M-PY - Beginner setup and worked micro-example

7. Follow with me once - fully worked teacher demonstration

Load sales and returns separately. State sales grain = one order line; returns grain = one return record/order in this fixture. Pick one order with multiple sales lines. Sum its line net_sales to one order total first. Read its single refund_amount. Compute $\text{order_net_after_refund} = \text{order_sales} - \text{refund}$. This tiny example proves why joining refunds directly to every line would multiply the refund.

8. What you should see / be able to say

You can show the original line rows, one order-level sales total, one refund and one order-level net-after-refund. The refund is subtracted once—not once per sales line.

9. Why it worked

Measures were brought to a compatible grain before combination. The same grain/cardinality discipline learned in SQL prevents plausible but inflated/deflated pandas results.

10. Common beginner mistakes

Merge returns directly to line-grain sales; drop dirty rows before logging; merge customer dimension despite duplicate cleaned key; use ``_merge`` then ignore `left_only`; calculate findings before totals reconcile.

11. If you get stuck - recovery ladder

Stop final charts/findings. Print shape, grain/key evidence and grouped totals at each stage. Isolate the first transformation/merge where rows or money change unexpectedly. Repair there, then rebuild downstream results.

M-PY - Full independent build and validation

12. Now you do the project

- Frame 2-4 business questions and exact metric definitions.
- Load all relevant source files into raw DataFrames; inspect shapes, types, missing, duplicate keys and value domains.
- Create copies for preparation. Detect duplicate line IDs, invalid discount/date and messy region; document rejection/correction rules.
- Build strict analysis set and reconcile its row count.
- Calculate line net_sales. Validate product key; merge products with `validate='many\_to\_one'` and `indicator=True`; keep unmatched visible.
- Resolve or quarantine the duplicate customer dimension key before any many-to-one customer enrichment.
- Build region/month/category summaries with explicit result grain and reconcile additive net sales.
- Aggregate sales to order grain before combining returns. Reconcile refund total and net-after-refunds.
- Create separate time/category/distribution figures with meaningful labels.
- Write three evidence-supported findings, one recommendation and limitations; save analysis table/figures with relative paths.
- Restart Kernel -> Run All. Fix the earliest failure. Save final clean-run evidence.

13. Validate

Required: strict row count; duplicate/invalid/unmatched controls; net-sales total; distinct orders;
top region check; returns count/refund total/net-after-refunds; at least one grouped total
reconciliation; saved outputs; clean Restart + Run All.

M-PY - Explain, evidence and project rubric

14. Teach it back in your own words

Give a two-minute defense: question -> source grains -> quality rules -> strict analysis set -> merge validation -> safe return grain -> grouped/visual analysis -> reconciliations -> strongest finding -> recommendation -> limitation -> clean-run proof.

15. Protected review + changed fresh retry

Save M-PY first. If one competency fails, open only the relevant protected lesson/project review. Close it and rebuild that stage on a changed question/data slice, then rerun downstream controls.

16. Evidence / mastery + exact next route

Critical pass: raw preserved; issues detected before correction; merge keys validated; unmatched visible; measures combined at compatible grain; grouped results reconcile; charts come from validated summaries; findings supported; limitation stated; outputs reproducible; Restart + Run All clean; learner can explain the notebook without review. PASS -> Gate A. PARTIAL/FAIL -> targeted repair + changed project evidence before Gate A.

End-of-module mastery checklist

- PY0: Explain notebook/kernel/hidden state and prove reproducible environment setup.
- PY1: Use clear variables/conditions/loop/function with exact boundary tests.
- PY2: Load/inspect/filter DataFrames before analysis.
- PY3: Detect before correcting; preserve issue evidence and row-count impact.
- PY4: Group/aggregate at stated result grain and reconcile additive totals.
- PY5: Prove dimension-key suitability; merge with validate= + indicator=True; investigate unmatched.
- PY6: Parse dates and reshape long/wide/long without changing underlying additive truth.
- PY7: Build separate purposeful Matplotlib/pandas figures from validated summaries.
- PY8: Perform compact EDA and separate quality findings, business observations and limitations.
- PY9: Structure outputs/paths and pass Restart Kernel -> Run All.
- PY10/M-PY: Build and defend an end-to-end analyst workflow with safe grain and validation.

Gate readiness

Attempt Gate A only when M-PY reruns cleanly and critical source/merge/total controls reconcile.

A clean Gate A pass ends M07; B/C are remediation full retests only.

Revision / Python quick reference

Load	<code>pd.read_csv(path)</code> ; inspect head/shape/dtypes/missing before analysis.
Dates	<code>pd.to_datetime(..., errors='coerce')</code> exposes invalid parse as NaT; it does not fix it.
Duplicates	<code>duplicated(..., keep=False)</code> shows all rows in duplicate group.
Filter	<code>df.loc[condition, columns]</code> keeps business scope explicit.
GroupBy	State grouped result grain; use size/nunique/sum/mean deliberately.
Merge	Prove key uniqueness; use <code>validate=</code> and <code>indicator=True</code> ; investigate <code>left_only</code> .
Reshape	<code>pivot/pivot_table</code> wide; <code>melt</code> long; reconcile totals.
Plots	Line=time, bar=category comparison, histogram=distribution; validate summary first.
EDA	Structure -> quality -> distributions -> groups -> trend -> findings -> limitations.
Output	Relative paths + <code>OUTPUT.mkdir(exist_ok=True) + to_csv(index=False)</code> .
Delivery	Restart Kernel -> Run All -> no hidden state -> saved evidence.

Next route: M-PY -> Gate A -> M08 Statistics only after Python competency pass.